



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

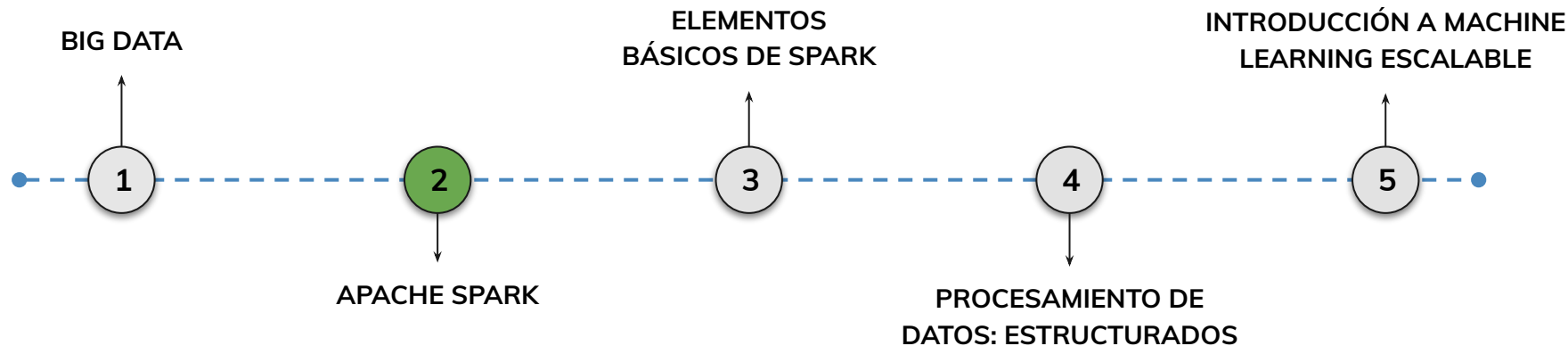


› Apache spark- Parte I

Aprendizaje Esperado 2: Explicar las características y componentes principales del framework apache spark para el procesamiento de altos volúmenes de datos.

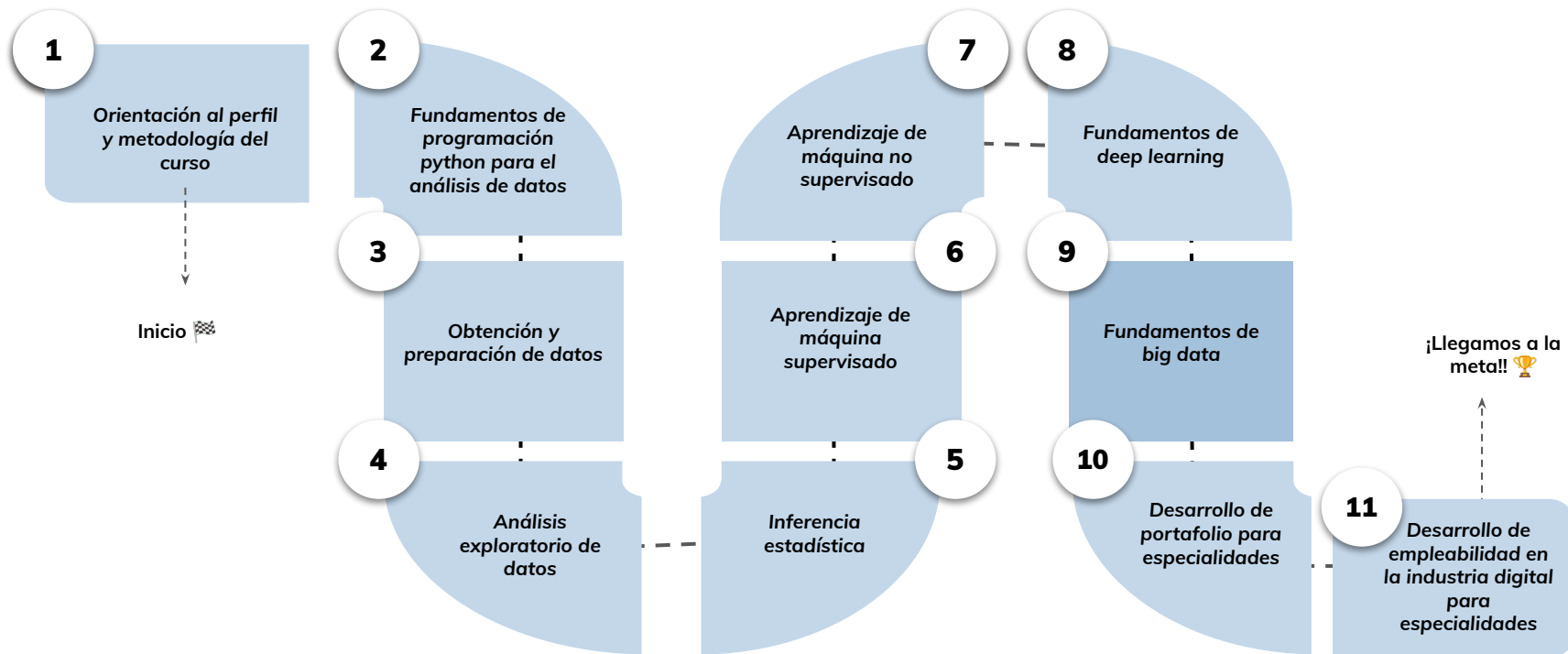
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



Learning Path

¿Cuáles temas trabajaremos hoy?

2.

Procesamiento distribuido con Apache Spark

Exploraremos los fundamentos de Apache Spark, su arquitectura distribuida, su ecosistema unificado y los problemas que resuelve frente a sistemas tradicionales como Hadoop MapReduce.

¿Qué es Apache Spark y por qué se necesita?

Introducción a Spark y diferencias con Hadoop.

Características clave: procesamiento en memoria, versatilidad, APIs.

Fundamentos técnicos de Apache Spark

RDD, DataFrames, Datasets.

Lazy evaluation y SparkContext.



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender qué es Apache Spark y cómo difiere de Hadoop MapReduce.
- Conocer las ventajas de Spark en términos de rendimiento, escalabilidad y flexibilidad.
- Identificar sus componentes clave: RDD, DataFrames, Datasets, SparkContext.
- Explicar cómo Spark gestiona el procesamiento en memoria y en tiempo real.
- Reconocer cuándo y por qué conviene utilizar Spark en un entorno distribuido.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Profundizamos en los conceptos clave de Big Data y sus 5V's.
- Exploramos arquitecturas de sistemas distribuidos.
- Conocimos tecnologías del ecosistema Big Data como Hadoop, Spark, Kafka.
- Analizamos la integración con aprendizaje automático y cloud computing.

Apache spark

Apache Spark: Procesamiento Distribuido de Datos Masivos

En la actualidad, el análisis de datos masivos y el procesamiento rápido de información se han convertido en necesidades esenciales para empresas e instituciones. La demanda de herramientas que permitan trabajar con grandes volúmenes de datos de manera eficiente ha impulsado el desarrollo de tecnologías como **Apache Spark**.

Apache Spark es un motor de procesamiento distribuido diseñado para manejar grandes conjuntos de datos a alta velocidad. A diferencia de sistemas tradicionales como Hadoop MapReduce, Spark ofrece un rendimiento superior gracias a su capacidad de procesamiento en memoria y su modelo de programación sencillo y flexible.

¿Qué es Apache Spark?

Apache Spark es un motor de procesamiento de datos distribuido y de código abierto, diseñado para ejecutar cargas de trabajo de análisis de datos a gran escala de forma rápida y eficiente. Spark fue desarrollado originalmente en la Universidad de California, en el laboratorio AMPLab de Berkeley, y en 2014 se convirtió en un proyecto de la **Apache Software Foundation**.

Spark nació como una respuesta a las limitaciones que presentaba el modelo de procesamiento **MapReduce** de Hadoop. Si bien MapReduce fue un avance importante en el tratamiento de datos masivos, su enfoque basado en lectura y escritura a disco después de cada operación generaba una alta latencia y un bajo rendimiento, especialmente en flujos de trabajo iterativos y análisis complejos.

Características principales de Apache Spark

Procesamiento en memoria

La principal diferencia de Spark respecto a sus predecesores es su capacidad para realizar el procesamiento de datos **en memoria (in-memory)**. Esto significa que los datos son almacenados en la memoria RAM de los nodos del clúster mientras se ejecutan las operaciones, evitando costosos accesos al disco y logrando una mejora significativa en los tiempos de ejecución.

Modelo de programación flexible

Además de su alto rendimiento, Spark destaca por su **modelo de programación sencillo y flexible**. Ofrece una interfaz unificada que permite trabajar con diferentes lenguajes de programación, como **Scala, Java, Python y R**, facilitando su adopción por parte de desarrolladores y analistas de datos.

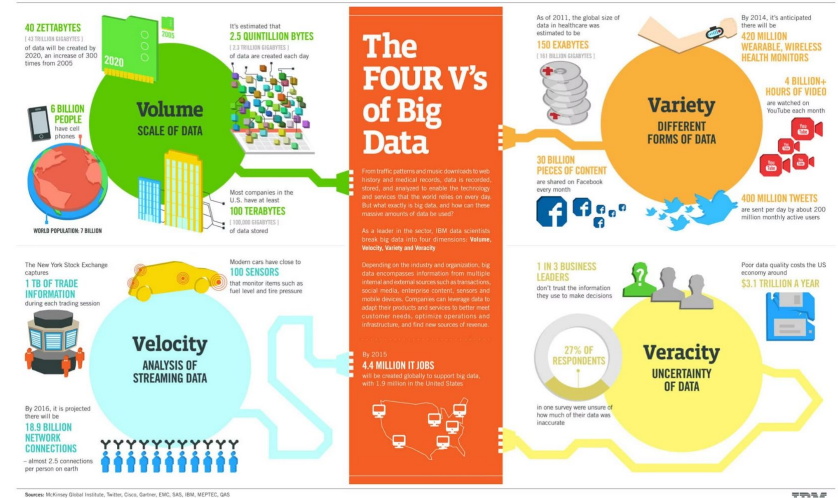
Versatilidad en cargas de trabajo

Otro aspecto importante de Spark es su capacidad para manejar diversos tipos de cargas de trabajo. No solo permite el procesamiento por lotes, sino también el análisis de flujos de datos en tiempo real, consultas SQL interactivas, aprendizaje automático y análisis de grafos, todo dentro de un mismo ecosistema.

Necesidad de Apache Spark

El crecimiento exponencial de los datos generados por empresas, redes sociales, sensores y dispositivos IoT ha incrementado la necesidad de soluciones eficientes y escalables para el análisis de datos masivos. Spark responde a esta necesidad ofreciendo una plataforma robusta, distribuida y de alto rendimiento.

En resumen, Spark es una herramienta esencial en el ecosistema de Big Data, ya que permite a las organizaciones procesar grandes volúmenes de datos de manera rápida y flexible, adaptándose a las demandas de análisis en tiempo real y procesamiento avanzado.

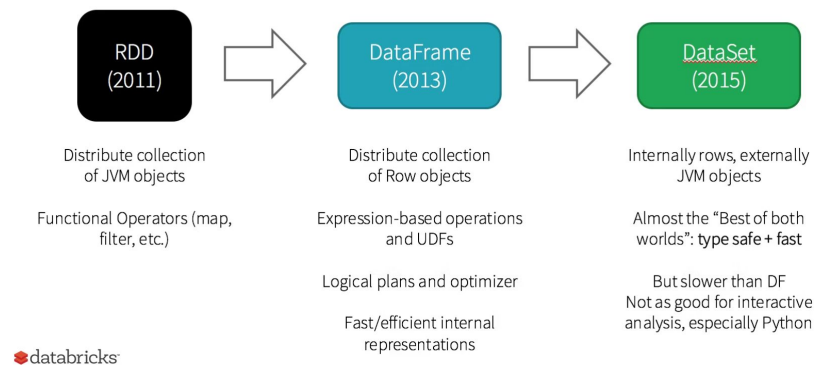


Conceptos básicos de Spark: RDD

Para comprender el funcionamiento de Apache Spark, es necesario conocer algunos conceptos básicos que forman la base de su arquitectura y operación. Estos conceptos permiten entender cómo Spark gestiona, procesa y distribuye los datos en un clúster.

Uno de los conceptos clave en Spark es el de **RDD (Resilient Distributed Dataset)**. Un RDD es una colección inmutable de objetos distribuidos a través de los nodos de un clúster. Los RDDs permiten realizar operaciones de transformación y acción de forma paralela, garantizando la tolerancia a fallos mediante un mecanismo llamado **lineage**, que permite reconstruir los datos en caso de pérdida.

History of Spark APIs



Conceptos básicos de Spark: DataFrames y Datasets

1

DataFrames

Los **DataFrames** son una colección distribuida de datos organizados en columnas, similar a una tabla en una base de datos relacional.

- Estructura tabular con nombres de columnas
- Optimizaciones automáticas mediante Catalyst Optimizer
- Interfaz similar a SQL para consultas

2

Datasets

Los **Datasets** combinan la eficiencia de los RDDs con la simplicidad de los DataFrames, ofreciendo tipado fuerte y validación en tiempo de compilación.

- Verificación de tipos en tiempo de compilación
- API orientada a objetos
- Mejor rendimiento que los RDDs tradicionales

Conceptos básicos de Spark: Evaluación perezosa

Otro concepto importante es el **Lazy Evaluation** o evaluación perezosa. En Spark, las transformaciones aplicadas sobre los RDDs, DataFrames o Datasets no se ejecutan inmediatamente, sino que se almacenan como un plan lógico que se optimiza y ejecuta cuando se solicita una acción, como contar elementos o recolectar resultados.

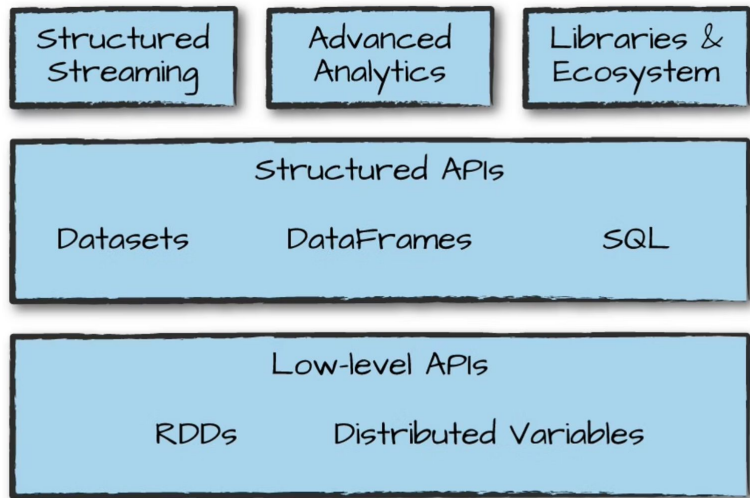


Figure 3-1. Spark's toolset

Este enfoque permite a Spark optimizar el plan de ejecución, minimizando el número de operaciones necesarias y mejorando el rendimiento general del procesamiento de datos.

Modelo de programación de Spark



Transformaciones

Las transformaciones, como `map()` o `filter()`, crean un nuevo RDD a partir de uno existente. Estas operaciones son perezosas y no se ejecutan hasta que se solicita una acción.

- `map()`
- `filter()`
- `groupByKey()`
- `join()`



Acciones

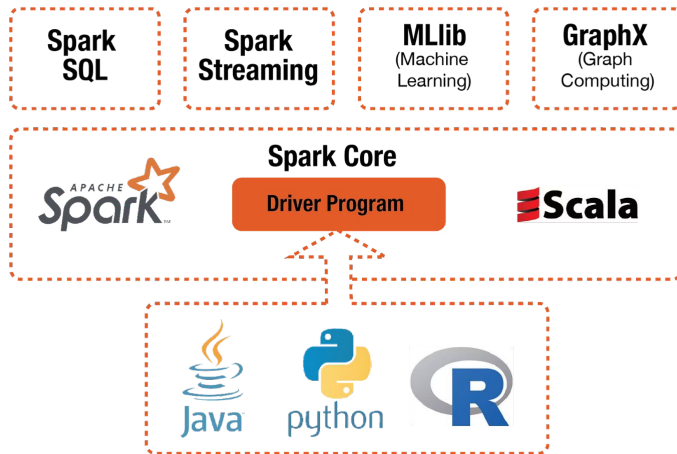
Las acciones, como `count()` o `collect()`, devuelven un resultado al programa o escriben datos en un sistema externo. Estas operaciones desencadenan la ejecución del plan de transformaciones.

- `count()`
- `collect()`
- `save()`
- `reduce()`

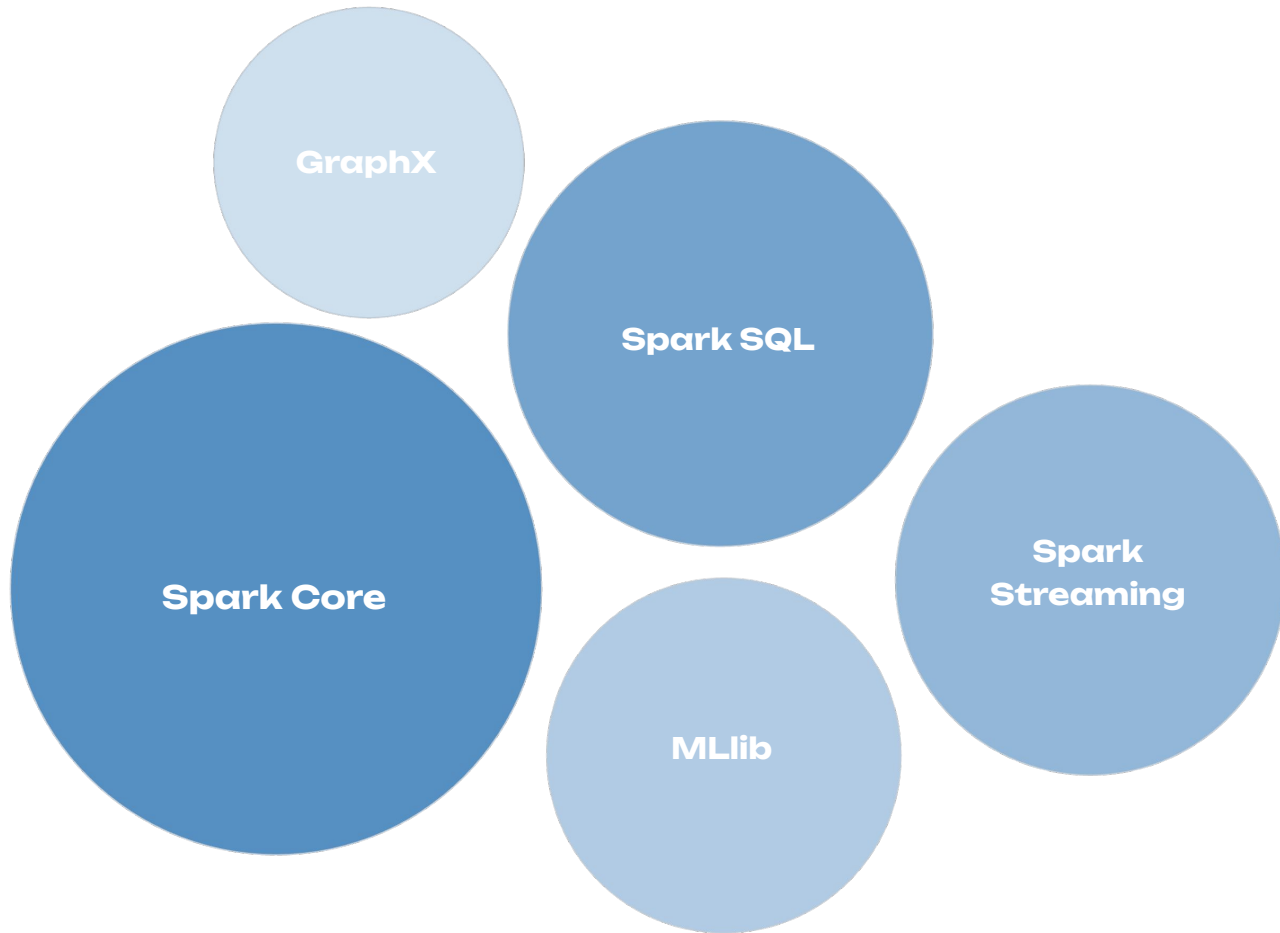
SparkContext y lenguajes de programación

Otro componente esencial en Spark es el **SparkContext**, que actúa como el punto de entrada principal para cualquier aplicación Spark. Este objeto coordina la distribución de datos y tareas a través del clúster.

Además, Spark permite el uso de múltiples **lenguajes de programación**. Si bien fue desarrollado inicialmente en Scala, ofrece APIs completas en **Python (PySpark)**, **Java** y **R**, lo que facilita su integración en diferentes entornos y perfiles técnicos.



Ecosistema unificado de Spark



Problemas que resuelve Spark: Latencia alta

Apache Spark fue creado para solucionar varios de los problemas que presentaban los sistemas tradicionales de procesamiento de datos masivos, principalmente aquellos basados en el paradigma MapReduce, como Hadoop.

Uno de los principales problemas que Spark resuelve es la **latencia alta en el procesamiento de datos**. Las soluciones basadas en MapReduce ejecutan cada operación de lectura y escritura sobre disco, lo que genera tiempos de espera elevados. Spark, en cambio, almacena los datos intermedios en memoria (RAM) durante las operaciones, reduciendo drásticamente los tiempos de ejecución.

Problemas que resuelve Spark: Análisis iterativos



Otro problema abordado por Spark es la **difícultad para realizar análisis iterativos**. En tareas como el aprendizaje automático o el análisis de grafos, es común que se requieran múltiples iteraciones sobre el mismo conjunto de datos. Con MapReduce, cada iteración implica leer y escribir datos en disco, mientras que Spark permite mantener los datos en memoria entre iteraciones, mejorando la eficiencia.

Problemas que resuelve Spark:

Procesamiento en tiempo real

Spark también resuelve la necesidad de **procesar datos en tiempo real**. Mientras que Hadoop MapReduce estaba diseñado únicamente para procesamiento por lotes, Spark incluye el módulo **Spark Streaming**, que permite el análisis de flujos de datos en tiempo real, facilitando casos de uso como la detección de fraudes, la monitorización de redes y la personalización de contenido.



Problemas que resuelve Spark: Fragmentación de herramientas

Antes de Spark

Las organizaciones debían combinar diferentes tecnologías para cubrir las necesidades de consultas SQL, aprendizaje automático y análisis de grafos.

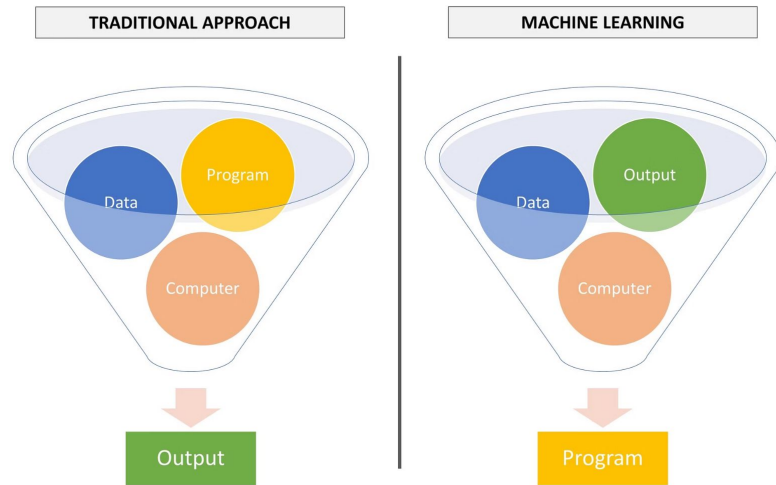
Con Spark

Spark unifica todas estas capacidades en un solo ecosistema, simplificando el desarrollo y la integración.

Problemas que resuelve Spark:

Complejidad en el modelo de programación

Otro problema común en los sistemas tradicionales es la **complejidad en el modelo de programación**. Spark ofrece un modelo más sencillo y flexible que MapReduce, con APIs intuitivas y soporte para varios lenguajes de programación, facilitando su adopción por parte de desarrolladores y analistas de datos.



Problemas que resuelve Spark:

Escalabilidad y recuperación ante fallos

Escalabilidad horizontal eficiente

La **falta de escalabilidad horizontal eficiente** era otro desafío que Spark abordó. Su arquitectura distribuida permite añadir nodos al clúster para incrementar la capacidad de procesamiento y almacenamiento de datos, garantizando un crecimiento flexible y controlado.

Recuperación ante fallos

Finalmente, Spark resuelve los problemas de **recuperación ante fallos** gracias a su modelo de datos basado en RDDs y el mecanismo de lineage, que permite reconstruir los datos en caso de que un nodo falle durante la ejecución de las tareas.

Comparativa con otros sistemas: Spark vs. Hadoop MapReduce

Modelo de procesamiento

La primera gran diferencia entre Spark y Hadoop MapReduce radica en el modelo de procesamiento. Mientras que **MapReduce** se basa en un procesamiento por lotes que lee y escribe en disco después de cada operación, **Spark utiliza un enfoque de procesamiento en memoria**. Esto le permite realizar las operaciones de manera mucho más rápida, reduciendo drásticamente la latencia y los tiempos de ejecución.

Versatilidad

Otra diferencia clave es la **versatilidad**. Hadoop MapReduce está limitado al procesamiento por lotes, mientras que Spark soporta diferentes tipos de cargas de trabajo en un solo ecosistema: procesamiento por lotes, consultas SQL interactivas, aprendizaje automático, procesamiento en tiempo real y análisis de grafos. Esto hace que Spark sea una solución más completa y flexible.

Facilidad de uso

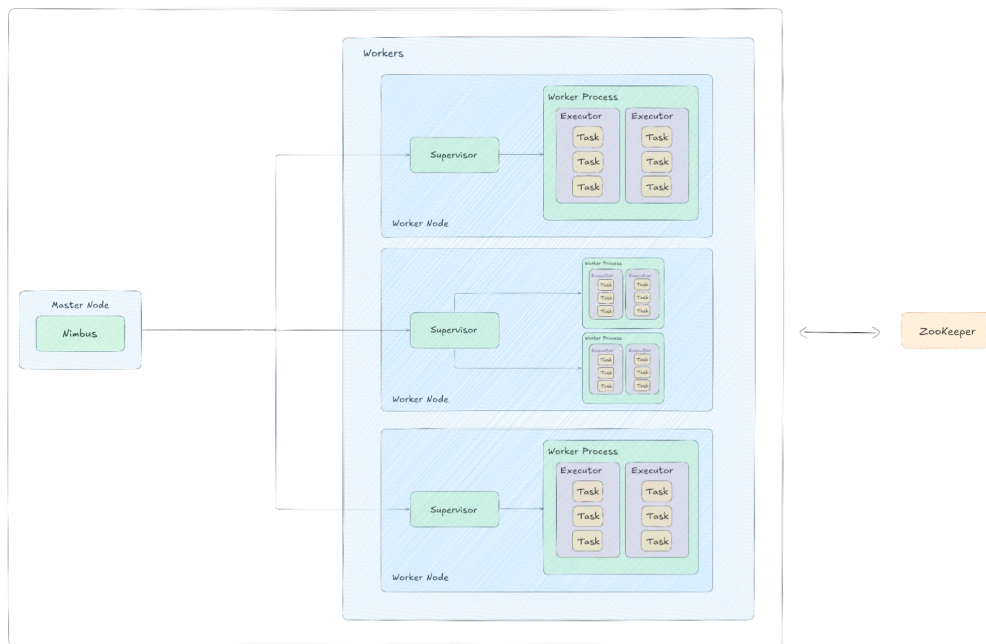
En cuanto a la facilidad de uso, Spark ofrece una **API más sencilla e intuitiva** que MapReduce. El modelo de programación de MapReduce es complejo y requiere escribir un gran volumen de código para tareas relativamente simples. Por el contrario, Spark proporciona APIs en múltiples lenguajes (Scala, Java, Python y R) y un modelo funcional que facilita el desarrollo.

Comparativa con otros sistemas: Spark vs. Apache Flink

Comparando Spark con **Apache Flink**, ambos sistemas permiten el procesamiento en tiempo real y por lotes. Sin embargo, Flink fue diseñado específicamente para flujos de datos continuos y ofrece una arquitectura **nativa para streaming**, mientras que Spark originalmente estaba orientado a procesamiento por lotes y añadió Spark Streaming como un módulo adicional.

Feature	Spark	Flink	Macrometa
Data Processing	Batch/Stream (Micro Batch)	Batch/Stream (Native)	Batch/Stream
Iterations	No	Yes	Yes
SQL	Spark SQL	Table, SQL API	SQL, Javascript
Fault tolerance	Yes (WAL)	Yes (Chandy-Lamport)	Yes (WAL)
Optimization	Manual	Auto	Auto
Windowing	Timed	Time, Count	Time, Count, Sliding, Tumbling
State Backend	HDFS	In-memory, file system, RocksDB	In-memory, multi-model DB
Language	Scala, Java, Python, R, C#, F#	Java, Scala, Python, SQL	SQL, Javascript
Cost	Open source	Open source	Commercial
Backpressure	Manual Configuration	Implicit, through architecture	Implicit
Geo-Distribution	No	No	Yes
Latency	Seconds	Sub-second	Sub-second
Geo-fencing	No	No	Yes
Converged Platform	No	No	Yes

Comparativa con otros sistemas: Spark vs. Apache Storm



Respecto a **Apache Storm**, este sistema también permite el procesamiento en tiempo real, pero está más enfocado en tareas de baja latencia y eventos individuales. Spark Streaming, en cambio, trabaja por micro-lotes, lo que puede introducir cierta latencia adicional, aunque permite una integración más sencilla con el resto del ecosistema Spark.

Comparativa: Tolerancia a fallos y rendimiento

Tolerancia a fallos

Otra diferencia relevante es la **gestión de tolerancia a fallos**. Tanto Spark como Hadoop y Flink cuentan con mecanismos robustos para recuperarse ante fallos de nodos o procesos. Spark utiliza su modelo de RDD y el concepto de lineage para reconstruir los datos, mientras que Flink ofrece un sistema de snapshots y Storm requiere configuración adicional para lograr tolerancia a fallos.

Rendimiento

En términos de **rendimiento**, múltiples estudios han demostrado que Spark supera a Hadoop MapReduce en la mayoría de los escenarios, especialmente en cargas de trabajo iterativas y de aprendizaje automático. En comparación con Flink y Storm, la elección dependerá de los requisitos específicos de latencia y volumen de datos.

Conclusión de la comparativa

En conclusión, Apache Spark se destaca por ofrecer un ecosistema unificado, alto rendimiento y flexibilidad para abordar diferentes tipos de procesamiento de datos, posicionándose como una de las soluciones más completas y adoptadas en el ámbito de Big Data.

Cuándo conviene usar Spark: Grandes volúmenes de datos

Apache Spark es una herramienta poderosa, pero no siempre es la solución ideal para todos los proyectos. Es importante conocer en qué situaciones y contextos resulta conveniente utilizar Spark para aprovechar al máximo sus capacidades y evitar un uso ineficiente de los recursos.

Spark es recomendable cuando se requiere **procesar grandes volúmenes de datos** que superan la capacidad de los sistemas tradicionales o de un solo servidor. Gracias a su arquitectura distribuida, Spark permite dividir y procesar datos masivos en múltiples nodos de un clúster, asegurando escalabilidad y eficiencia.



Cuándo conviene usar Spark: Análisis complejos e iterativos



Aprendizaje automático

Algoritmos que requieren múltiples pasadas sobre los mismos datos, como clasificación, regresión y clustering.



Análisis de grafos

Procesamiento de relaciones complejas entre entidades, como redes sociales o sistemas de recomendación.



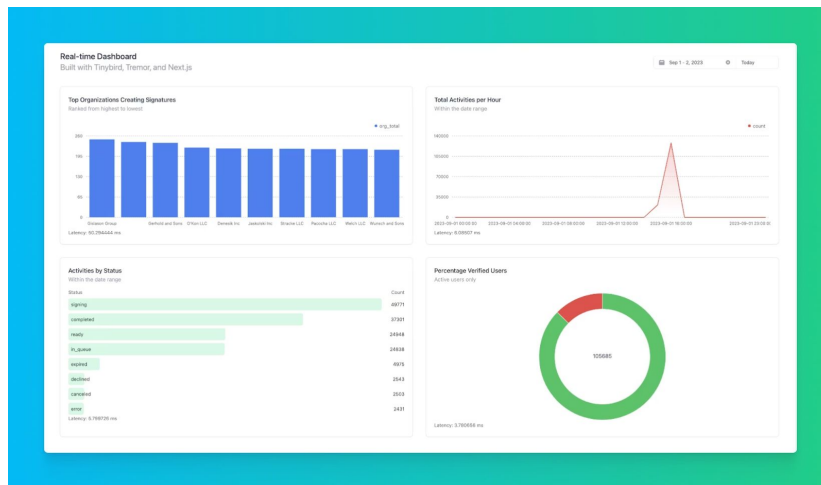
Minería de datos

Descubrimiento de patrones y extracción de conocimiento a partir de grandes conjuntos de datos.

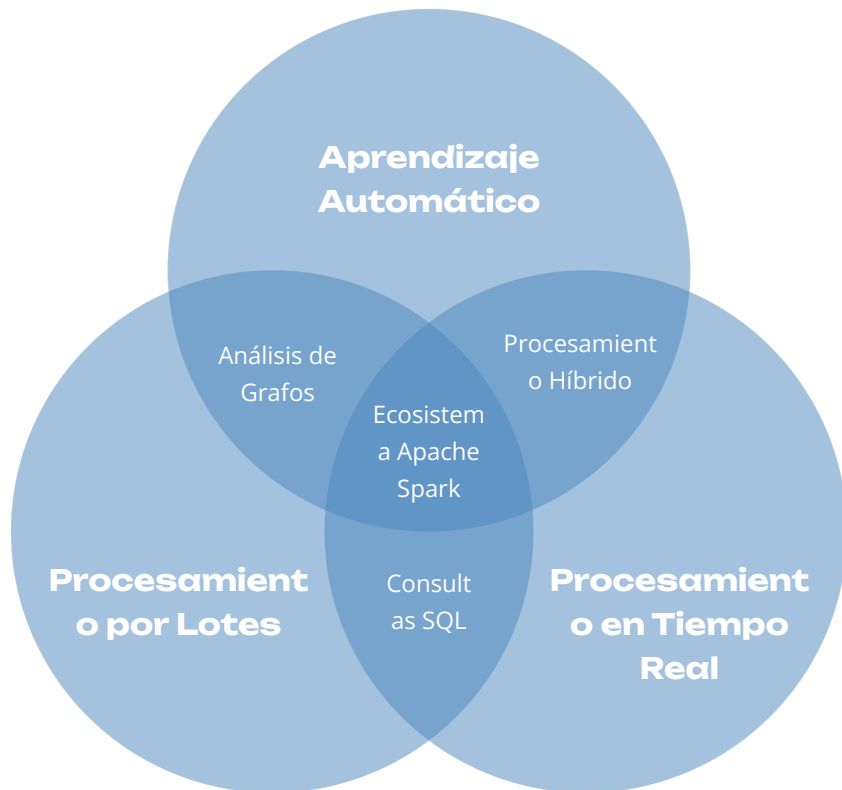
Otra situación ideal para utilizar Spark es cuando las tareas de procesamiento requieren **análisis complejos e iterativos**, como los algoritmos de aprendizaje automático y minería de datos. Spark, al trabajar con datos en memoria, reduce drásticamente los tiempos de ejecución frente a sistemas que dependen de la lectura y escritura en disco.

Cuándo conviene usar Spark: Procesamiento en tiempo real

El uso de Spark también es recomendable en entornos donde se necesita **procesamiento en tiempo real o cuasi tiempo real**. Mediante su módulo **Spark Streaming**, Spark permite capturar y procesar flujos de datos provenientes de sensores, redes sociales, sistemas financieros, entre otros, facilitando la toma de decisiones inmediata.



Cuándo conviene usar Spark: Ecosistema unificado



Cuando un proyecto requiere integrar diferentes tipos de procesamiento —como consultas SQL, aprendizaje automático, procesamiento de datos estructurados y no estructurados, y análisis de grafos— Spark es una opción adecuada, ya que ofrece un ecosistema unificado para cubrir todas estas necesidades.

Cuándo conviene usar Spark: Experiencia del equipo y entornos distribuidos

Experiencia del equipo

Spark es especialmente útil en proyectos donde el equipo de trabajo ya tiene experiencia en lenguajes como **Scala, Java, Python o R**, ya que su API soporta estos lenguajes y facilita el desarrollo de soluciones personalizadas.

Entornos distribuidos

En entornos donde la infraestructura es distribuida, como clústeres locales, servidores en la nube o sistemas híbridos, Spark es una herramienta adecuada gracias a su capacidad de adaptarse y aprovechar los recursos disponibles de manera eficiente.

Cuándo NO conviene usar Spark

Sin embargo, Spark no es la mejor opción para proyectos con conjuntos de datos pequeños o medianos que pueden ser procesados fácilmente con herramientas tradicionales o bases de datos relacionales. En estos casos, el despliegue y gestión de un clúster Spark puede resultar excesivo y costoso.



Live Coding

¿En qué consistirá la Demo?

Vamos a investigar el funcionamiento de Apache Spark

¿Qué investigaremos y explicaremos?

1. ¿Qué sucede cuando ejecutas un script en Spark?
 - a. Rol del Driver y los Workers.
 - b. El papel del SparkContext y del Cluster Manager (YARN, Standalone, Mesos, Kubernetes).
 - c. ¿Cómo se transforma un DAG en tareas distribuidas?
2. ¿Qué significa que Spark use “evaluación diferida” (lazy evaluation)?
 - a. ¿Qué ventajas tiene sobre otros frameworks?
 - b. Diferencia entre transformations y actions.
3. Comparación entre RDD, DataFrame y Dataset
 - a. ¿Cuándo conviene usar cada uno?
 - b. Cuadro comparativo: rendimiento, flexibilidad, facilidad de uso.
4. Explora uno de estos módulos y explica para qué sirve
 - a. Spark SQL, MLlib, GraphX o Spark Streaming.



Momento:

Time-out!



5 -10 min.





Ejercicio

Explorando Apache Spark

Explorando Apache Spark

Contexto: 🙌

Vas a interiorizar los conceptos clave de Apache Spark mediante una exploración guiada de sus componentes y casos de uso.

Consigna: 📝

1. Explica con tus palabras qué es Apache Spark y qué lo diferencia de MapReduce.
2. Describe los conceptos de RDD, DataFrame y Dataset y señala sus diferencias.
3. Enumera los módulos que componen el ecosistema Spark y para qué se usa cada uno.
4. Investiga tres casos reales o simulados donde Spark sea más conveniente que otras herramientas.

Paso a paso: ⚙️

1. Usa el material de clase como guía base.
2. Resume cada bloque de contenido en tus propias palabras.
3. Organiza los resultados en un cuadro comparativo o infografía.
4. Prepara una breve presentación (oral o visual) para compartir en la próxima clase.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ **Comprendimos qué es Apache Spark y su rol en el procesamiento de datos masivos.**
- ✓ **Identificamos sus ventajas clave frente a Hadoop: procesamiento en memoria, menor latencia y mayor eficiencia.**
- ✓ **Conocimos su arquitectura técnica: RDD, DataFrames, Datasets y SparkContext.**
- ✓ **Exploramos el ecosistema Spark: Spark SQL, Streaming, MLlib, GraphX.**
- ✓ **Analizamos comparativas con otras herramientas como Flink, Storm y Hadoop MapReduce.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

